

CH4 – Clip 7.

Orchestrator 이론



01

Orchestrator란?

03

Orchestrator 예시 및 MLOps에서의 활용

02

Orchestrator 주요 기능

04

Appendix : YAML이란?

1. Orchestrator 란?

Orchestrator 란?

정의

- 다수의 Container를 조정하고 관리하는 시스템. 컨테이너의 배포, 스케일링 및 네트워킹을 자동화

목적

- 높은 가용성, 확장성 및 신뢰성을 가진 시스템을 구축하기 위해, Container화된 애플리케이션의 복잡한 작업을 간소화

Orchestrator 중요성

복잡성 관리

- Orchestrator는 복잡한 서비스의 생명 주기를 관리하고, 여러 Container 간의 의존성을 조정

자동화

- 자동화를 통해 수동 intervention 없이도 시스템을 안정적으로 운영 가능

확장성

- 사용량 증가에 따라 자동으로 리소스를 조정하고, 필요에 따라 서비스를 확장하거나 축소할 수 있음

고가용성

- 서비스의 지속적인 가용성을 보장하며, 장애 발생 시 자동으로 복구해줌

2. Orchestrator 주요 기능

Orchestrator 주요 기능

주요 기능

- 자동 배포 및 관리
- 스케일링 및 로드 밸런싱
- 자동 복구 및 장애 대응
- 서비스 발견 및 네트워킹
- 업데이트 및 롤백 관리

Orchestrator 주요 기능

자동 배포 및 관리

- Orchestrator는 사용자가 정의한 설정에 따라 Container를 자동으로 배포하고, 실행
- 세부 내용
사용자는 Container Image, 환경 변수, 실행할 명령어 등을 저으이
Orchestrator는 이러한 정의에 따라 Container를 생성하고, 네트워크 설정과 볼륨 마운트 등을 처리

스케일링 및 로드 밸런싱

- 트래픽의 증가나 감소에 따라 자동으로 컨테이너 수를 조절
- 세부 내용
사용량이 많은 시간에는 컨테이너 수를 늘려 부하를 분산시키고, 사용량이 적은 시간에는 줄여 리소스를 효율적으로 사용
들어오는 요청을 여러 컨테이너에 균등하게 분배하여 전체 시스템의 균형을 유지

Orchestrator 주요 기능

자동 복구 및 장애 대응

- 오류가 발생한 컨테이너를 자동으로 감지하고, 필요한 경우 복구 조치를 취함
- 세부 내용
Container가 실패하거나 비정상적인 상태로 동작할 경우, Orchestrator는 해당 Container를 자동으로 재시작하거나 새 instance로 교체
시스템의 안정성과 가용성을 유지하기 위한 조치를 자동으로 실행

서비스 발견 및 네트워킹

- Orchestrator는 컨테이너 간의 통신 및 서비스 발견을 관리
- 세부 내용
내부 DNS 또는 다른 서비스 발견 메커니즘을 통해 Container 간의 통신을 용이하게 함
네트워크 정책을 통해 컨테이너 간의 통신을 제어하고 보안을 강화

Orchestrator 주요 기능

업데이트 및 롤백 관리

- 애플리케이션 업데이트를 안전하게 관리하고, 필요한 경우 이전 버전으로 롤백
- 세부 내용
 - 업데이트 중 발생할 수 있는 다운타임을 최소화하기 위해 롤링 업데이트와 같은 전략을 사용
 - 업데이트가 실패하거나 문제가 발생할 경우, 자동으로 이전 버전으로 롤백하여 시스템 안정성을 보장

3. Orchestrator 활용 사례

Orchestrator 활용 사례

대규모 머신러닝 파이프라인 관리

- **배경** : 대형 기술 회사가 다양한 소스에서 수집된 대량의 데이터를 처리하고, 여러 머신러닝 모델을 효율적으로 트레이닝하고 배포하기 위한 파이프라인을 구축할 필요성이 있음
- **오케스트레이터 활용**: Kubernetes를 사용하여 데이터 전처리, 모델 트레이닝, 평가 및 배포를 포함하는 전체 파이프라인을 자동화
- **결과**: 모델 개발 시간 단축, 효율적인 리소스 관리 및 배포, 높은 가용성 및 확장성을 달성

실시간 데이터 처리 및 분석

- **배경** : 금융 서비스 회사가 실시간으로 금융 시장 데이터를 분석하고, 이를 통해 트레이딩 결정을 지원하는 머신러닝 기반 시스템을 구축할 필요성 있음
- **오케스트레이터 활용**: Kubernetes 클러스터를 활용하여 실시간 데이터 스트림을 처리하고, 빠른 의사결정을 위한 분석 모델을 실행
- **결과**: 데이터 처리 지연 시간 감소, 높은 처리량 및 시스템의 안정적인 운영을 달성

Orchestrator 활용 사례

다양한 머신러닝 모델의 동시 트레이닝 및 배포

배경: E-commerce 플랫폼이 사용자의 행동을 기반으로 개인화된 추천을 제공하기 위해 여러 머신러닝 모델을 개발 및 유지보수 필요함

- **오케스트레이터 활용:** 다양한 머신러닝 모델을 Kubernetes 클러스터 내에서 동시에 트레이닝하고, 이를 효율적으로 배포 및 관리
- **결과:** 높은 확장성과 유연성을 가진 모델 트레이닝 및 배포 환경 구축, 빠른 모델 업데이트 및 성능 최적화 달성

대규모 분산 모델 트레이닝

- **배경:** 의료 연구 기관이 복잡한 의료 이미지를 분석하는 데 사용되는 딥러닝 모델을 대규모 데이터셋에서 트레이닝 필요
- **오케스트레이터 활용:** Kubernetes를 활용하여 대규모의 GPU 리소스를 분산 배치하고, 효율적인 모델 트레이닝 파이프라인을 구축
- **결과:** 높은 성능의 모델 트레이닝, 리소스 활용 최적화 및 시간 단축 달성

Orchestrator의 MLOps에서 중요성

복잡한 워크플로우 관리

- 머신러닝 프로젝트는 데이터 수집, 전처리, 모델 트레이닝, 평가, 배포 등 다양한 단계로 구성. Orchestrator는 이러한 과정들을 자동화하여, 수동으로 작업을 조정하는 복잡성과 오류 가능성을 줄임
- 자동 스케일링 및 로드 밸런싱 기능을 통해 리소스 사용을 최적화하고, 필요에 따라 유연하게 리소스를 할당하거나 회수

고가용성 및 복구 메커니즘

- 시스템의 가용성을 높이고 장애가 발생했을 때 빠르게 복구
- 머신러닝 시스템의 신뢰성 높임

CI/CD

- 머신러닝 모델의 지속적인 통합과 배포를 지원. 모델이 지속적으로 개선되고 새로운 데이터에 신속하게 적응할 수 있도록 도움

4. YAML 이란?

YAML 이란?

정의

- **YAML : YAML Ain't Markup Language**
- 데이터 직렬화 언어
- 구성 파일, 데이터 교환, 설정 파일 등에 널리 사용됨
- YAML은 읽기 쉽고, 쓰기 쉬우며, 프로그래밍 언어 간 데이터 교환에 적합

특징

- **가독성** : JSON이나 XML에 비해 인간이 읽기 쉬움
- **계층적 구조** : 들여쓰기를 사용하여 데이터의 계층 구조를 표현
- **다양한 데이터 타입 지원** : 스칼라 타입(문자열, 숫자, boolean), 리스트, 맵 등을 지원
- **멀티 도큐먼트 지원** : 하나의 파일 내에서 여러 문서를 '---'로 구분하여 표현할 수 있음

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: nginx
      ports:
        - containerPort: 80
```


YAML 이란?

주의 사항

- **들여 쓰기** : 공백을 사용한 들여쓰기가 중요 (탭 사용을 피해야함)
- **데이터 타입** : 숫자, Boolean 등의 데이터 타입이 자동으로 변환될 수 있으므로 주의 필요

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: nginx
      ports:
        - containerPort: 80
```

YAML 이란?

기준	YAML	XML	JSON
전체 명칭	YAML Ain't Markup Language	eXtensible Markup Language	Javascript Object Notation
가독성	최고	좋음. 하지만 태그가 많아 복잡할 수 있음	매우 좋음
구문	키-값 쌍, 들여쓰기	태그 기반	키-값 쌍
데이터 타입 지원	다양함	제한적	다양함
주석 지원	제한적	지원함	지원하지 않음
파일 크기	작은편	큼	중간
파싱 속도	중간	느림	빠름
주 사용 사례	설정 파일, 간단한 데이터 교환	복잡한 문서 구조, 웹서비스	웹 API, 설정 파일

YAML 이란?

```
person:
  name: John Doe
  age: 30
  married: true
  children:
    - name: Jane Doe
      age: 10
    - name: Jack Doe
      age: 8
```

```
<person>
  <name>John Doe</name>
  <age>30</age>
  <married>true</married>
  <children>
    <child>
      <name>Jane Doe</name>
      <age>10</age>
    </child>
    <child>
      <name>Jack Doe</name>
      <age>8</age>
    </child>
  </children>
</person>
```

```
{
  "person": {
    "name": "John Doe",
    "age": 30,
    "married": true,
    "children": [
      {
        "name": "Jane Doe",
        "age": 10
      },
      {
        "name": "Jack Doe",
        "age": 8
      }
    ]
  }
}
```

YAML 이란?

```
person:
  name: John Doe
  age: 30
  married: true
  children:
    - name: Jane Doe
      age: 10
    - name: Jack Doe
      age: 8
```

YAML

```
<person>
  <name>John Doe</name>
  <age>30</age>
  <married>true</married>
  <children>
    <child>
      <name>Jane Doe</name>
      <age>10</age>
    </child>
    <child>
      <name>Jack Doe</name>
      <age>8</age>
    </child>
  </children>
</person>
```

XML

```
{
  "person": {
    "name": "John Doe",
    "age": 30,
    "married": true,
    "children": [
      {
        "name": "Jane Doe",
        "age": 10
      },
      {
        "name": "Jack Doe",
        "age": 8
      }
    ]
  }
}
```

JSON

YAML 이란?

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: nginx
      ports:
        - containerPort: 80
```

kubernetes

```
stages:
  - build
  - test

build_job:
  stage: build
  script:
    - echo "Building the project..."
    - build_script.sh

test_job:
  stage: test
  script:
    - echo "Running tests..."
    - test_script.sh
```

GitLab CI